

Conference Planning System

Scenario

You have been asked to create the data structure for a small technology conference.

The conference has multiple sessions, speakers, rooms, topics and registered participants. Your task is to represent and manage this information using the Python collection types covered in Lesson 2.

Unlike the standard lab, this challenge does not tell you exactly which Python collection to use for every part of the system.

You are expected to decide how the data should be structured and be able to explain your choices.

Important: Only use concepts covered so far in the course. You do not need loops, functions, classes or file handling for this challenge.

Part 1 – Design the conference data

Create a conference with at least:

- 8 conference sessions
- 5 speakers
- 4 rooms
- 10 registered participants
- 5 different topics/categories

Each session should contain relevant information such as:

- session title
- speaker
- room
- start time
- duration
- topic/category
- maximum number of participants

Some information belongs naturally together, some information may change during the conference, and some information should ideally be unique.

Choose suitable Python collections to represent the data.

Your solution must make meaningful use of:

- lists
- dictionaries
- tuples
- sets

- nested collections

You should not create separate unrelated examples just to demonstrate each collection type. The collections should be part of the conference system.

Part 2 – Work with the schedule

Using the conference structure you created, demonstrate that you can retrieve information from nested data.

Without using loops:

1. Print the title of the first session.
2. Print the speaker of the third session.
3. Print the room of the last session.
4. Print all information about one selected session.
5. Print one specific value from a nested structure using chained indexing.
6. Print the first three sessions.
7. Print the last two sessions.
8. Create a reversed version of the session schedule using slicing.
9. Create a copy containing only part of the schedule.

Your code should access the existing conference data rather than manually writing the same information again.

Part 3 – Conference changes

The conference schedule changes during the day.

Modify your existing data to reflect the following events:

- One session changes room.
- One speaker is replaced by another speaker.
- A new session is added.
- One session is cancelled and removed.
- One participant registers for the conference.
- One participant cancels their registration.
- One session receives an additional piece of information called difficulty, with a value such as "Beginner", "Intermediate" or "Advanced".

Print enough information before and after the changes to demonstrate that your data was actually modified.

Part 4 – Unique conference information

The organizers want to keep track of unique information.

Use the data in your system to represent:

- unique conference topics

- unique technical skills represented by speakers
- participants registered for two different workshops

For the two workshops, determine:

- participants registered for both workshops
- participants registered only for Workshop A
- participants registered only for Workshop B
- all unique participants registered for either workshop

Choose collection operations that make these comparisons straightforward.

Part 5 – Conference configuration

Some values describe fixed information that belongs together and should not normally be modified individually.

Create suitable structures for at least three of the following:

- conference dates
- opening and closing times
- room coordinates or floor/location information
- conference contact information
- session time slots

Access individual values from these structures and unpack at least one of them into separate variables.

Be prepared to explain why your chosen collection is appropriate for this kind of information.

Part 6 – The shared-reference problem

The organizers want to create a backup of the participant list before making changes.

Start with your existing participant list and create:

```
backup_participants = participants
```

Then modify backup_participants.

Print both variables.

Explain in a comment:

- What happened?
- Why did changing the backup also affect the original list?

Now create the backup in a way that allows you to modify the new list without modifying the original list.

Demonstrate that your solution works.

Extra challenge

Investigate what happens if the list contains dictionaries and you use:

```
participants.copy()
```

Change a value inside one of the dictionaries in the copied list.

Does the original data change?

Explain what you observe.

You do not need to solve this using techniques we have not covered yet, but you should be able to explain the behaviour.

Part 7 – Restructure the data

The conference organizers currently have some information stored separately:

```
session_titles = [
    "Python for AI",
    "Building APIs",
    "Introduction to LLMs"
]

speakers = [
    "Ada",
    "Grace",
    "Alan"
]

rooms = [
    "Room A",
    "Room B",
    "Room C"
]
```

This works, but it becomes difficult to understand which values belong together as the conference grows.

Create a better structure that connects each session with its corresponding speaker and room.

Then demonstrate how you can access:

- the second session's title
- its speaker
- its room

Add at least three additional pieces of information to each session.

In a short comment, explain why your new structure is easier to work with than several separate lists.

Final Challenge – Build the Complete Conference State

Bring your work together into one final structure representing the current state of the conference.

Your final solution should make it possible to find information about:

- the conference itself
- sessions
- speakers
- rooms
- participants
- topics
- workshop registrations

The information should be logically connected rather than stored as many unrelated variables.

For example, someone reading your code should be able to understand relationships such as:

conference → sessions → speaker

and retrieve a specific value through the structure.

Demonstrate your final design by retrieving and printing at least 10 different pieces of information from different levels of the structure.

At least five of these must require accessing nested data.

Then make at least five changes to the conference state using appropriate collection operations.

Design explanation

At the bottom of your Python file, answer the following questions using comments:

10. Where did you use a list, and why was a list suitable?
11. Where did you use a dictionary, and why was a dictionary suitable?
12. Where did you use a tuple, and why was a tuple suitable?
13. Where did you use a set, and why was a set suitable?
14. Which parts of your data are mutable?
15. Which parts should ideally remain unchanged?
16. What is one advantage of using nested collections?
17. What is one disadvantage of deeply nested collections?
18. What is the difference between assigning one list to another variable and copying the list?
19. If you were allowed to use concepts from later lessons, what part of your solution would you most want to improve?

Requirements

Your solution must:

- use all four main collection types covered in the lesson: list, tuple, set and dict
- use nested collections in a meaningful way
- demonstrate indexing
- demonstrate negative indexing

- demonstrate slicing
- demonstrate unpacking
- demonstrate adding, updating and removing data
- demonstrate set operations
- demonstrate the difference between a shared list reference and a copied list
- use descriptive variable names
- contain comments explaining important design decisions
- run without errors

You should be able to explain why you chose each data structure, not only how the syntax works.

Restrictions

For this challenge, do not use:

- loops
- functions
- classes
- file handling
- external libraries

These concepts will be introduced or developed in later lessons.

The challenge is to build and manipulate a reasonably complex data model using only the Python concepts we have covered so far.